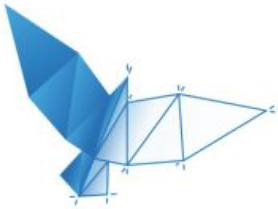

# NVISO Labs

**Cyber security research, straight from the lab!** 🐀

Menu

# Intercept Flutter traffic on iOS and Android (HTTP/HTTPS/Dio Pinning)

Jeroen Beckers Android, iOS, Flutter, Frida August 18, 2022 7 Minutes

Some time ago I wrote some articles on how to Man-In-The-Middle Flutter on iOS, Android (ARM) and Android (ARM64). Those posts were quite popular and I often went back to copy those scripts myself.

Last week, however, we received a Flutter application where the script wouldn't work anymore. As we had the source code, it was easy to figure out that the application was using the dio package to perform SSL Pinning.

While it would be possible to remove the pinning logic and recompile the app, it's much nicer if we can just disable it at runtime, so that we don't have to recompile ourselves. The result of this post is a Frida script that works both on Android and iOS, and disables the full TLS verification including the pinning logic.

## TL;DR

- Get the latest version of the script from NVISOSecurity/disable-flutter-tls-verification, or use Frida CodeShare
- If it fails, share your app so the memory pattern can be improved
- Flutter still doesn't listen to Proxy settings, so use ProxyDroid on Android or a VPN on iOS

## The test app

As usual, we'll create a test app to validate our script. I've created a basic Flutter app similar to the previous posts which has three buttons: HTTP, HTTPS and HTTPS (Pinned).

The app can be found on the GitHub page and an APK and IPA build are available. The Dio pinning logic is pretty straightforward:

```
ByteData data = await rootBundle.load('raw/certificate.
Dio dio = Dio();
(dio.httpClientAdapter as DefaultHttpClientAdapter).onH
  SecurityContext sc = new SecurityContext();
  sc.setTrustedCertificatesBytes(data.buffer.asUint8Lis
  HttpClient httpClient = new HttpClient(context: sc);
  return httpClient;
};

try {
  Response response = await dio.get("https://www.nviso.
  _status = "HTTPS: SUCCESS (" + response.headers.value
} catch (e) {
  print("Request via DIO failed");
  print("Exception: $e");
  _status = "DIO: ERROR";
}
```

## The new approach

Originally, we hooked the ssl_crypto_x509_session_verify_cert_chain function, which can currently be found at line 361 of ssl_x509.cc. This method is responsible for validating the certificate chain, so if this method returns true, the certificate chain must be valid and the connection is accepted.

When performing a MitM on the test app on Android ARM64, the following error is printed in logcat:

```
3540  3585 I flutter : Request via DIO failed
3540  3585 I flutter : Exception: DioError [DioErrorType.other]:
HandshakeException: Handshake error in client (OS Error:
3540  3585 I flutter :    CERTIFICATE_VERIFY_FAILED: self signed
certificate in certificate chain(handshake.cc:393))
3540  3585 I flutter : Source stack:
3540  3585 I flutter : #0      DioMixin.fetch
(package:dio/src/dio_mixin.dart:488)
3540  3585 I flutter : #1      DioMixin.request
(package:dio/src/dio_mixin.dart:483)
3540  3585 I flutter : #2      DioMixin.get
(package:dio/src/dio_mixin.dart:61)
3540  3585 I flutter : #3      _MyHomePageState.callPinnedHTTPS
(package:flutter_pinning_demo/main.dart:124)
3540  3585 I flutter : <asynchronous suspension>
3540  3585 I flutter : HandshakeException: Handshake error in client (OS
Error:
3540  3585 I flutter :    CERTIFICATE_VERIFY_FAILED: self signed
certificate in certificate chain(handshake.cc:393))
```

Flutter gives us some nice information: there's a self-signed certificate in the certificate chain, which it doesn't like.

The original MitM script hooks session_verify_cert_chain, and for some reason the hooks were never triggered. The session_verify_cert_chain method is called from ssl_verify_peer_cert on line 386 and the error that is shown above results from OPENSSL_PUT_ERROR on line 393:

```
366 uint8_t alert = SSL_AD_CERTIFICATE_UNKNOWN;
367   enum ssl_verify_result_t ret;
368   if (hs->config->custom_verify_callback != nullptr) {
369     ret = hs->config->custom_verify_callback(ssl, &ale
370     switch (ret) {
```

```
371         case ssl_verify_ok:
372           hs->new_session->verify_result = X509_V_OK;
373           break;
374         case ssl_verify_invalid:
375           // If |SSL_VERIFY_NONE|, the error is non-fata
376           if (hs->config->verify_mode == SSL_VERIFY_NONE
377             ERR_clear_error();
378             ret = ssl_verify_ok;
379           }
380           hs->new_session->verify_result = X509_V_ERR_AP
381           break;
382         case ssl_verify_retry:
383           break;
384       }
385     } else {
386       ret = ssl->ctx->x509_method->session_verify_cert_c
387                 hs->new_session.get(), hs, &alert)
388                 ? ssl_verify_ok
389                 : ssl_verify_invalid;
390     }
391
392     if (ret == ssl_verify_invalid) {
393       OPENSSL_PUT_ERROR(SSL, SSL_R_CERTIFICATE_VERIFY_FA
394       ssl_send_alert(ssl, SSL3_AL_FATAL, alert);
395     }
```

The code path that is most likely taken, is that a `custom_verify_callback` is registered, which makes line 368 return true, and the callback executed on line 369 returns `ssl_verify_invalid`. The code then jumps to line 392 and the `ret` variable does equal `ssl_verify_invalid` so the alert is shown.

```
366 uint8_t alert = SSL_AD_CERTIFICATE_UNKNOWN;
367   enum ssl_verify_result_t ret;
368   if (hs->config->custom_verify_callback != nullptr) {
369     ret = hs->config->custom_verify_callback(ssl, &ale
370     switch (ret) {
371       case ssl_verify_ok:
372         hs->new_session->verify_result = X509_V_OK;
373         break;
374       case ssl_verify_invalid:
375         // If |SSL_VERIFY_NONE|, the error is non-fata
376         if (hs->config->verify_mode == SSL_VERIFY_NONE
377           ERR_clear_error();
378           ret = ssl_verify_ok;
379         }
380         hs->new_session->verify_result = X509_V_ERR_AP
381         break;
382       case ssl_verify_retry:
383         break;
384     }
385   } else {
386     ret = ssl->ctx->x509_method->session_verify_cert_c
```

```
                hs->new_session.get(), hs, &alert)
                ? ssl_verify_ok
                : ssl_verify_invalid;
    }

    if (ret == ssl_verify_invalid) {
      OPENSSL_PUT_ERROR(SSL, SSL_R_CERTIFICATE_VERIFY_FA
      ssl_send_alert(ssl, SSL3_AL_FATAL, alert);
    }
```

The easiest approach would be to hook the ssl_verify_peer_cert function and modify the return value to be ssl_verify_ok, which is 0. By hooking this earlier method, both the default SSL validation and any custom validation is disabled. Unfortunately, the ssl_send_alert function already triggers an error and so modifying the return value of ssl_verify_peer_cert would be too late.

Fortunately, we can just throw out the entire function and replace it with a `return 0` statement:

```
function hook_ssl_verify_peer_cert(address)
{
    Interceptor.replace(address, new NativeCallback((pat
        console.log("[+] Certificate validation disabled
        return 0;
    }, 'int', ['pointer', 'int']));
}
```

The only thing that's left is finding the actual location of the ssl_verify_peer_cert function.

## Finding the offsets

### Manually

The approach which was explained in the previous blogposts can be followed to identify the ssl_verify_peer_cert function:

- Find references to the string "x509.cc" and compare them to x509.cc to find session_verify_cert_chain
- Find references to the method you identified in order to identify ssl_verify_peer_cert

Both x509.cc and handshake.cc use the OPENSSL_PUT_ERROR macro which swaps in the file name and line number, which you can use to identify the correct functions.

### By pattern matching

Alternatively, we can use Frida's pattern matching engine to search for functions that look very similar to the function from the demo app. The first bytes of a function are typically very stable, as long as the number of local variables and function arguments don't change. Still, different compilers may generate different assembly code (e.g. usage of different registers or optimisations) so we do need to have some wildcards in our pattern.

After downloading and creating multiple Flutter apps with different Flutter versions, I came to the following list:

> iOS x64: FF 83 01 D1 FA 67 01 A9 F8 5F 02 A9 F6 57 03 A9 F4 4F 04 A9 FD 7B 05 A9 FD 43 01 91 F? 03 00 AA 1? 00 40 F9 ?8 1A 40 F9 15 ?5 4? F9 B5 00 00 B4
> Android x64: F? 0F 1C F8 F? 5? 01 A9 F? 5? 02 A9 F? ?? 03 A9 ?? ?? ?? ?? 68 1A 40 F9
> Android x86: 2D E9 FE 43 D0 F8 00 80 81 46 D8 F8 18 00 D0 F8 ?? 71

These patterns should only result in one hit in the libFlutter library and all match to the start of the ssl_verify_peer_cert function.

## The final script

Putting all of this together gives the following script. It's one script that can be used on Android x86, Android x64 and iOS x64.

> **Check GitHub for the latest version**
>
> The script below may have been updated on the GitHub repo.

```
var TLSValidationDisabled = false;
```

```
var secondRun = false;
if (Java.available) {
    console.log("[+] Java environment detected");
    Java.perform(hookSystemLoadLibrary);
    disableTLSValidationAndroid();
    setTimeout(disableTLSValidationAndroid, 1000);
} else if (ObjC.available) {
    console.log("[+] iOS environment detected");
    disableTLSValidationioOS();
    setTimeout(disableTLSValidationioOS, 1000);
}

function hookSystemLoadLibrary() {
    const System = Java.use('java.lang.System');
    const Runtime = Java.use('java.lang.Runtime');
    const SystemLoad_2 = System.loadLibrary.overload('
    const VMStack = Java.use('dalvik.system.VMStack');

    SystemLoad_2.implementation = function(library) {
        try {
            const loaded = Runtime.getRuntime().loadLi
            if (library === 'flutter') {
                console.log("[+] libflutter.so loaded"
                disableTLSValidationAndroid();
            }
            return loaded;
        } catch (ex) {
            console.log(ex);
        }
    };
}

function disableTLSValidationioOS() {
    if (TLSValidationDisabled) return;

    var m = Process.findModuleByName("Flutter");

    // If there is no loaded Flutter module, the setTi
    if (m === null) {
        if (secondRun) console.log("[!] Flutter module
        secondRun = true;
        return;
    }

    var patterns = {
        "arm64": [
            "FF 83 01 D1 FA 67 01 A9 F8 5F 02 A9 F6 57
        ],
    };
    findAndPatch(m, patterns[Process.arch], 0);

}

function disableTLSValidationAndroid() {
    if (TLSValidationDisabled) return;

```

```
    var m = Process.findModuleByName("libflutter.so");

    // The System.loadLibrary doesn't always trigger,
    if (m === null) {
        if (secondRun) console.log("[!] Flutter module
        secondRun = true;
        return;
    }

    var patterns = {
        "arm64": [
            "F? 0F 1C F8 F? 5? 01 A9 F? 5? 02 A9 F? ??
        ],
        "arm": [
            "2D E9 FE 43 D0 F8 00 80 81 46 D8 F8 18 00
        ]
    };
    findAndPatch(m, patterns[Process.arch], Process.ar
}

function findAndPatch(m, patterns, thumb) {
    console.log("[+] Flutter library found");
    var ranges = m.enumerateRanges('r-x');
    ranges.forEach(range => {
        patterns.forEach(pattern => {
            Memory.scan(range.base, range.size, patter
                onMatch: function(address, size) {
                    console.log('[+] ssl_verify_peer_c
                    TLSValidationDisabled = true;
                    hook_ssl_verify_peer_cert(address.
                }
            });
        });
    });

    if (!TLSValidationDisabled) {
        if (secondRun)
            console.log('[!] ssl_verify_peer_cert not
        else
            console.log('[!] ssl_verify_peer_cert not
    }
    secondRun = true;
}

function hook_ssl_verify_peer_cert(address) {
    Interceptor.replace(address, new NativeCallback((p
        return 0;
    }, 'int', ['pointer', 'int']));
}
```

## About the author

## Jeroen Beckers

Jeroen Beckers is a mobile security expert working in the NVISO Software Security Assessment team. He is a SANS instructor and SANS lead author of the SEC575 course. Jeroen is also a co-author of OWASP Mobile Security Testing Guide (MSTG) and the OWASP Mobile Application Security Verification Standard (MASVS). He loves to both program and reverse engineer stuff.

LinkedIn

**Tagged:** android, flutter, ios, frida, pinning

**Published by Jeroen Beckers**

*View all posts by Jeroen Beckers*

‹ Finding hooks with windbg

Cortex XSOAR Tips & Tricks – Creating indicator relationships in integrations ›

# 11 thoughts on “Intercept Flutter traffic on iOS and Android (HTTP/HTTPS/Dio Pinning)”

> Pingback: Intercept Flutter traffic on iOS and Android (HTTP/HTTPS/Dio Pinning) – NVISO Labs – Library 11: Antigonish Project Edition

> Pingback: Intercepting Flutter traffic on iOS – NVISO Labs

> Pingback: Intercepting Flutter traffic on Android (ARMv8) – NVISO Labs

> Pingback: Intercepting traffic from Android Flutter applications – NVISO Labs

> Pingback: We're celebrating our 10th anniversary! – NVISO Labs

**josé**

June 9, 2023 at 8:42 am

apparently something changed in the last version(s) and this isn't working on new builds anymore…. either that or I'm doing something wrong. in any case, this is what I get for ssl_verify_result() (I assume, I hope I'm not wrong…) in an androir app I'm pentesting:
FUN_006a3cb8 XREF[4]: 002f8550, 0038a71c(*),
FUN_00753530:0075387c(c),
FUN_00753530:007545fc(c)
006a3cb8 ff 43 01 d1 sub sp,sp,#0x50
006a3cbc fe 13 00 f9 str x30,[sp, #local_30]
006a3cc0 f6 57 03 a9 stp x22,x21,[sp, #local_20]
006a3cc4 f4 4f 04 a9 stp x20,x19,[sp, #local_10]
006a3cc8 e1 02 00 b4 cbz x1,LAB_006a3d24
but even then, the hook isn't working… yet.

Loading...

↩ Reply

**Jeroen Beckers**

June 9, 2023 at 10:13 am

Hi Jose! Can you open a ticket on the github repo?
https://github.com/NVISOsecurity/disable-flutter-tls-verification

Loading...

↪ Reply

**josé**

June 12, 2023 at 6:21 am

Nevermind, I did everything wrong… It's working for my app. I had some tool not work and messed up the analysis/RE.

Loading...

Pingback: iOS Android Flutter HTTP/HTTPS/Dio Pinning -

Pingback: Unpacking Flutter hives – NVISO Labs

Pingback: Unpacking Flutter hives - F1TYM1

# Leave a Reply

NVISO Homepage

Jobs

**Info and support**

info@nviso.eu

**Got hacked?**

csirt@nviso.eu